The simple Primitive equation model

Consider the Boussinesq equations in hydrostatic approximation

$$\partial_t \boldsymbol{u} + f\underset{\neg}{\boldsymbol{u}} + \boldsymbol{\nabla} p = -\boldsymbol{u}\cdot\boldsymbol{\nabla}\boldsymbol{u} - w\partial_z \boldsymbol{u} + \boldsymbol{F}(\boldsymbol{u}) \quad , \quad \partial_t b = -\boldsymbol{u}\cdot\boldsymbol{\nabla} b - w\partial_z b + Mb \tag{1}$$

with the diagnostic relations $\partial_z p = b$ and $\boldsymbol{\nabla}\cdot\boldsymbol{u} + \partial_z w = 0$. Vectors are two-dimensional and $\underset{\neg}{\boldsymbol{u}}$ denotes anticlockwise rotation by 90°. Pressure is scaled with background density ρ_0 and thermodynamics have been simplified to a buoyancy equation. $\boldsymbol{F}$ and M are the friction and mixing operators. At the free surface $z = \eta$, we also have the condition

$$\partial_t p_s = -g\boldsymbol{\nabla}\cdot\int_{-H}^{\eta} \boldsymbol{u} dz \tag{2}$$

with $p_s = p|_{z=\eta} = g\eta$. Alternatively, we use a rigid lid with $w = 0$ at $z = 0$ and $p_s = p|_{z=0}$. We split full pressure p into hydrostatic part and surface pressure.

$$p = p_h(z) + p_s \ , \ p_h = \int_{-h}^{z} b dz' \tag{3}$$

where p_s is to be determined by the external mode solvers.

Simple implicit external solver for rigid lid

Velocity and buoyancy and pressure are all co-located in time in this scheme. The buoyancy equation is integrated as

$$b^{n+1} - b^n = \Delta t \left(\overline{\delta b}^{n+1/2} + M(b^n) \right) \tag{4}$$

with the advective buoyancy tendency

$$\delta b^n = -\boldsymbol{\nabla}\cdot\boldsymbol{u}^n b^n - \partial_z w^n b^n \quad , \quad \overline{\delta b}^{n+1/2} = A\delta b^n + B\delta b^{n-1} + C\delta b^{n-2} \tag{5}$$

interpolated with the Adam-Bashforth (AB3) scheme to time level $n+1/2$. The biharmonic horizontal mixing (attribute `Khbi`) is treated with an Euler forward step. Vertical mixing follows the options described in the section on vertical mixing and friction below.

An intermediate velocity $\boldsymbol{u}^*$ is calculated from

$$\boldsymbol{u}^* - \boldsymbol{u}^n = \Delta t \left(\overline{\delta \boldsymbol{u}}^{n+1/2} + \boldsymbol{F}(\boldsymbol{u}^n) \right) \tag{6}$$

with the momentum tendency $\delta\boldsymbol{u}$ excluding the surface pressure gradient and friction

$$\delta\boldsymbol{u}^n = -f\underset{\neg}{\boldsymbol{u}}^n - \boldsymbol{\nabla} p_h^n - \boldsymbol{u}^n\cdot\boldsymbol{\nabla}\boldsymbol{u}^n - w^n\partial_z\boldsymbol{u}^n \tag{7}$$

These tendencies are extrapolated with AB3 to time level $n + 1/2$

$$\overline{\delta \boldsymbol{u}}^{n+1/2} = A\delta\boldsymbol{u}^n + B\delta\boldsymbol{u}^{n-1} + C\delta\boldsymbol{u}^{n-2} \tag{8}$$

Friction is treated with an Euler forward step. From the vertically integrated continuity equation follows the condition

$$\boldsymbol{\nabla} \cdot \int_{-h}^{0} \boldsymbol{u}^* = \Delta t \boldsymbol{\nabla} \cdot h \boldsymbol{\nabla} p_s^{n+1} \tag{9}$$

which is solved with a conjugate-gradient iteration for p_s^{n+1} (attributes `epsilon` for the convergence threshold and `max_iterations`; a BiCGStab variant is implemented as well and is selected by editing the solver call in `pressure.py`). The iteration count and the estimated error are available per step as `congr_itts` and `estimated_error`. The intermediate velocity is then corrected as

$$\boldsymbol{u}^{n+1} = \boldsymbol{u}^* - \Delta t \boldsymbol{\nabla} p_s^{n+1} \tag{10}$$

It is possible to include an implicit free surface in the time stepping. Vertical velocity is calculated from continuity

$$w^{n+1} = -\int_{-h}^{z} \boldsymbol{\nabla} \cdot \boldsymbol{u}^{n+1} \tag{11}$$

This scheme is also used as default in the MITgcm.

Simple split-explicit external mode solver

Velocity is known in this scheme at $t = n + 1/2$, while b and p at level n. The momentum equation is discretised as

$$\boldsymbol{u}^* - \boldsymbol{u}^{n-1/2} = \Delta t \left(-f \underline{\overline{\boldsymbol{u}}}^{n} + \overline{\delta \boldsymbol{u}}^{n} - \boldsymbol{\nabla} p_h^n - \boldsymbol{\nabla} p_s^n + \boldsymbol{F}(\boldsymbol{u}^{n-1/2}) \right) \tag{12}$$

with the intermediate velocity $\boldsymbol{u}^*$ and AB3 extrapolation of Coriolis force and advection terms tendencies

$$\underline{\overline{\boldsymbol{u}}}^{n} = A \underline{\boldsymbol{u}}^{n-1/2} + B \underline{\boldsymbol{u}}^{n-3/2} + C \underline{\boldsymbol{u}}^{n-5/2} \quad , \quad \overline{\delta \boldsymbol{u}}^{n} = A \delta \boldsymbol{u}^{n-1/2} + B \delta \boldsymbol{u}^{n-3/2} + C \delta \boldsymbol{u}^{n-5/2} \tag{13}$$

and

$$\delta \boldsymbol{u}^{n-1/2} = -\boldsymbol{u}^{n-1/2} \cdot \boldsymbol{\nabla} \boldsymbol{u}^{n-1/2} - w^{n-1/2} \partial_z \boldsymbol{u}^{n-1/2} \tag{14}$$

For the hydrostatic and surface pressure $p_h + p_s$ no extrapolation is made, since they are at level n anyways, and friction is treated with Euler forward.

Now the momentum tendencies are vertically integrated but without surface pressure and Coriolis contributions.

$$\boldsymbol{R}^n = \int_{-h}^{0} \left(\overline{\delta \boldsymbol{u}}^{n} - \boldsymbol{\nabla} p_h^n + \boldsymbol{F}(\boldsymbol{u}^{n-1/2}) \right) dz \tag{15}$$

The external mode is integrated with sub-cycling over $m = 1, ...M$ (M set by the attribute `sub_cycling`) from n to $n+1$ with a simple, dissipative scheme proposed by Demange et al (2019) using the forcing $\boldsymbol{R}^n$

$$\boldsymbol{U}^{n+(m+1)/M} - \boldsymbol{U}^{n+m/M} = \Delta t/M \left(-f \overline{\underline{\boldsymbol{U}}}^{n+(m+1/2)/M} - h \boldsymbol{\nabla} p_s^{n+m/M} + \boldsymbol{R}^n \right) \quad (16)$$

$$p_s^{n+(m+1)/M} - p_s^{n+m/M} = -g \Delta t/M \left((1+\theta) \boldsymbol{\nabla} \cdot \boldsymbol{U}^{n+(m+1)/M} - \theta \boldsymbol{\nabla} \cdot \boldsymbol{U}^{n+m/M} \right) \quad (17)$$

with the transport $\boldsymbol{U} = \int_{-h}^{0} \boldsymbol{u}\, dz$, and $\overline{\boldsymbol{U}}^{n+(m+1/2)/M}$ interpolated with AB2. Demange et al (2019) found $\theta = 0.14$ to be the best choice in terms of phase error and damping. In the model θ is the attribute `fb_theta` with default 0.14. Larger values damp grid-scale barotropic waves more strongly.

The sub cycle starts from the transport $\boldsymbol{U}^n$ and the surface pressure p_s^n which the sub cycle of the previous time step ended with. This transport is model state (`U_bt`, `V_bt`) and part of the restart. It is computed from the vertical integral of $\boldsymbol{u}^n$ only at a cold start. Two transports result from the sub cycle. The transport at the end of the sub cycle, $\boldsymbol{U}^{n+1} = \boldsymbol{U}^{n+M/M}$, starts the next sub cycle. The average over the sub cycle

$$\tilde{\boldsymbol{U}}^{n+1} = \frac{1}{M} \sum_{m=1}^{M} \boldsymbol{U}^{n+m/M} + \frac{\theta}{M} \left(\boldsymbol{U}^{n+1} - \boldsymbol{U}^n \right) \quad (18)$$

is used to correct the barotropic part of the intermediate velocity

$$\boldsymbol{u}^{n+1/2} = \boldsymbol{u}^* - 1/h \left(\int_{-h}^{0} \boldsymbol{u}^* dz - \tilde{\boldsymbol{U}}^{n+1} \right) \quad (19)$$

The next sub cycle must not start from the average $\tilde{\boldsymbol{U}}^{n+1}$. That would pair the averaged transport with the surface pressure at the end of the sub cycle. This mismatch grows from step to step when the Courant number of the external gravity wave is large (Banerjee et al. 2024, the split-explicit solver of FESOM2). The buoyancy equation is integrated using $\boldsymbol{u}^{n+1/2}$ and $w^{n+1/2} = -\int_{-h}^{z} \boldsymbol{\nabla} \cdot \boldsymbol{u}^{n+1/2}$ as

$$b^{n+1} - b^n = \Delta t \left(\delta b^{n+1/2} + M(b^n) \right) \quad (20)$$

with the buoyancy tendency

$$\delta b^n = -\boldsymbol{\nabla} \cdot \boldsymbol{u}^{n+1/2} b^n - \partial_z w^{n+1/2} b^n \quad (21)$$

interpolated with AB3 to time level $n + 1/2$. This scheme is one option in FESOM2.

In both schemes the advective tracer flux through the surface face is zero by default, as in pyOM2. Under the linear free surface the vertical velocity w at the surface face is not zero. A tracer that is uniform in space then changes in the surface cell at the rate $b\, w|_{z=0}/\Delta z = -b\, \boldsymbol{\nabla} \cdot \int_{-h}^{0} \boldsymbol{u}\, dz/\Delta z$. With the attribute `free_surface_tracer_flux` the flux through the surface face is $w\, b$ instead, with w at the surface face and b of the surface cell. A uniform tracer then stays uniform. This option is off by default. The external-mode treatment is selected by the attributes `implicit_free_surface` and `explicit_free_surface`: both false gives the rigid lid, the first the implicit free surface (both solved with the iterative solver above), the second the split-explicit scheme.

Tracer advection

A tracer T lives at the centre of a cell. The advective flux through a face of the cell is the velocity through the face times a value T_f of the tracer at the face. The face between cell i and cell $i+1$ in one direction is used below. The other directions are treated in the same way. Three choices of T_f are available.

The default is the mean of the two cells, $T_f = (T_i + T_{i+1})/2$. This value is of second order. It conserves the variance of the tracer. It produces under- and overshoots at fronts.

With `superbee_advection` the face value is the mean minus a correction from the Superbee flux limiter, as in MITgcm. The correction contains the Courant number $|u|\Delta t/\Delta x$. In a forward step the scheme is monotone. Inside the Adams–Bashforth step with the weights 1.6 and -0.6 it is stable up to a Courant number of about 0.45. It sharpens fronts and flattens smooth extrema.

With `weno_advection` the face value is the fifth-order WENO reconstruction of Oceananigans. The code is the module `advection_weno.py` in `pyOM3`. For a flow from cell i to cell $i+1$ the five cells $i-2$ to $i+2$ are used. Three parabolas through three neighbouring cells each give a value at the face,

$$p_0 = \frac{2T_{i-2} - 7T_{i-1} + 11T_i}{6} \ , \quad p_1 = \frac{-T_{i-1} + 5T_i + 2T_{i+1}}{6} \ , \quad p_2 = \frac{2T_i + 5T_{i+1} - T_{i+2}}{6} \ .$$

The face value is $T_f = \omega_0 p_0 + \omega_1 p_1 + \omega_2 p_2$ with weights ω_r that sum to one. For a flow from cell $i+1$ to cell i the cells $i-1$ to $i+3$ are used in mirrored order.

The weights are

$$\omega_r = \frac{\alpha_r}{\alpha_0 + \alpha_1 + \alpha_2} \ , \quad \alpha_r = C_r \left(1 + \left(\frac{\tau}{\beta_r + \epsilon} \right)^2 \right)$$

with $C_0 = 1/10$, $C_1 = 6/10$ and $C_2 = 3/10$. β_r is the smoothness of parabola r after Jiang and Shu (1996). $\tau = |\beta_0 - \beta_2|$ is the global smoothness after Borges et al. (2008). Where T is smooth all β_r are nearly equal, the weights are close to C_r and T_f is of fifth order. Where the three cells of a parabola cross a jump its β_r is large and its weight falls to zero.

β_r is of the size of the squared difference ΔT^2 between neighbouring cells. It is compared with the constant $\epsilon = 10^{-8}\, T_{ref}^2$. T_{ref} is given by `weno_ref` in the unit of the tracer. The weights therefore react only to differences larger than about $10^{-4}\, T_{ref}$. Only differences enter, so the mean value of the tracer plays no role. A salinity of 35 does not ask for $T_{ref} = 35$.

T_{ref} should be the size of the jumps and fronts of interest, up to about three times that. Where a jump is much smaller than T_{ref}, all β_r are small against ϵ. The weights then stay at C_r and the scheme oscillates at the jump like a linear fifth-order scheme. Where a feature is about as large as T_{ref} or larger, the weights react to it. A smooth feature only a few cells wide is then treated like a jump and its peak is cut.

Table 1 shows both effects. A jump of size 1 and Gaussians of amplitude 1 were advected once over 200 cells, with AB3 and the Courant number 0.3. The over- and undershoot at

T_{ref}	excursion at the jump	peak, width 6 cells	peak, width 12 cells
0.1	$8 \cdot 10^{-6}$	0.73	0.999
1	$3 \cdot 10^{-5}$	0.61	0.999
3	$1 \cdot 10^{-4}$	0.98	0.999
10	$4 \cdot 10^{-4}$	0.98	0.999
100	$4 \cdot 10^{-3}$	0.98	0.999
10^4	$6 \cdot 10^{-2}$	—	—

Table 1: A jump of size 1 and Gaussians of amplitude 1 advected once over 200 cells with the WENO scheme, for different T_{ref}. The width of a Gaussian is its value of σ in cells. The last row is buoyancy of order 10^{-4} m/s^2 with the default $T_{ref} = 1$.

the jump grows in proportion to T_{ref}. The Gaussian with a width of 6 cells is cut for T_{ref} up to the amplitude. The Gaussian with a width of 12 cells does not depend on T_{ref}.

Table 2 gives values of T_{ref} for common cases. The same T_{ref} is used for temperature and salinity. $T_{ref} = 1$ suits both in an ocean model. If unsure, take the largest difference between neighbouring cells in the initial field, or the range across a front. A T_{ref} wrong by a factor of 3 does no harm. A T_{ref} too large by a factor of 100 or more makes the scheme oscillate at jumps.

tracer	typical jump	T_{ref}
temperature in degC, ocean fronts	0.1 to 5 K	1 (default)
salinity in psu	0.1 to 1	1 (default)
temperature 10 and 12 degC in overflow1_3D	2 K	2
buoyancy as tracer (`eq_of_state` = 0)	10^{-3} to 10^{-2} m/s^2	10^{-2}
buoyancy anomaly of a dimensional LES	10^{-4} to 10^{-3} m/s^2	10^{-4} to 10^{-3}
non-dimensional buoyancy	of order 1	1 (default)
dye between 0 and 1	1	1
age in years	10 to 100	30 to 100

Table 2: Values of T_{ref} (`weno_ref`, in nh_simple `weno_ref_b`) for common cases.

Next to land not all five cells are wet. If the three cells $i - 1$ to $i + 1$ are wet, the third-order WENO value of these three cells is used. Otherwise the value of the upstream cell is used, $T_f = T_i$. The wet cells are taken from the mask of the T cells. The surface face is therefore treated like a wall. Oceananigans uses the mean of the two cells in this last case. In overflow1_3D this mean produced undershoots of half a jump next to land, where Superbee stays bounded. The upstream value removes them. Each face has one flux, so the content of the tracer is conserved in all cases.

The scheme needs three halo points, `halo_points` = 3. Inside the Adams–Bashforth step its Courant number may reach about 0.40 with AB3 and 0.45 with the weights 1.6 and −0.6. The mean of the two cells allows 0.72 and 0.50. The scheme is not monotone.

In the tests its excursions beyond the initial range stay below 10^{-4} of a jump. At a single sharp front Superbee is sharper and cheaper. The WENO scheme pays where the tracer is smooth.

If both switches are set, the WENO scheme is used.

Vertical mixing and friction options

Vertical mixing of buoyancy and vertical friction are organised in a hierarchy of options similar to pyOM2, selected by the attribute `vmix_scheme` $\in$ {`'explicit'`, `'implicit'`} and two switches (module `implicit_vmix.py`; the previous flag `enable_implicit_vmix` maps to the implicit scheme with convective adjustment, so existing configurations are unchanged).

Explicit scheme. Fixed coefficients κ (buoyancy) and A_v (momentum) only, integrated forward in time,

$$b^{n+1} = b^n + \Delta t\, \partial_z \left(\kappa\, \partial_z b^n\right), \qquad \boldsymbol{u}^{n+1} = \boldsymbol{u}^{n+1} + \Delta t\, \partial_z \left(A_v\, \partial_z \boldsymbol{u}\right) \tag{22}$$

subject to the diffusive stability limit, which is checked at start-up (`check_vmix_stability`, called from `prepare_for_run`): $4 \max(\kappa, A_v)\, \Delta t/\Delta z^2 \leq 1/2$; the model refuses to run otherwise, and also refuses convective adjustment or the TKE closure under the explicit scheme.

Implicit scheme. Both operators are integrated with backward Euler (unconditionally stable, so arbitrarily large coefficients are allowed), as a tridiagonal solve per column with the coefficient located on the W points. Three sub-options for the coefficients: (i) constant κ and A_v; (ii) convective adjustment (`enable_conv_adjust`): background κ, but $\kappa = \kappa_{conv}$ (large) where $N^2 < 0$; (iii) the TKE closure (`enable_tke`), which overrides all constant coefficients for both buoyancy and momentum as described below. Vertical friction is applied after the surface pressure correction: since the vertical operator with zero-flux boundary conditions leaves the column-integrated momentum unchanged, the barotropic mode is not perturbed, and w is re-diagnosed afterwards.

Bottom friction. Two options act on the velocity $\boldsymbol{u}_b$ in the lowest wet cell of each U and V column. Both are off by default. Linear friction (`enable_bottom_friction`) has the rate $r = r_{bot}$ in 1/s (attribute `r_bot`). Quadratic friction has the rate $r = C_d |\boldsymbol{u}_b| / \Delta z$ with the drag coefficient C_d (attribute `r_quad_bot`). It is switched on by `enable_quadratic_bottom_friction`. Here $|\boldsymbol{u}_b|$ is the speed in the bottom cell. The other velocity component is averaged to the U or V point from the four surrounding points. If both options are on, the two rates add. The friction is an implicit step on the bottom cell,

$$\boldsymbol{u}_b^{new} = \boldsymbol{u}_b / \left(1 + \Delta t\, r\right)$$

In the rigid-lid and implicit free-surface scheme it is applied after the vertical friction. In the split-explicit scheme it is applied before the external mode is integrated. There the same bottom stress also acts on the transport inside the sub cycle. Every sub step divides $\boldsymbol{U}$ by $1 + (\Delta t/M)\, r\, \Delta z/h$.

TKE closure

The closure of Gaspar et al. (1990) is ported from pyOM2 (`tke.py`, mixin class). All closure constants are configuration attributes. They are set before `init_tke`. The defaults are the pyOM2 `tke_module` defaults $c_k = 0.1$, $c_\epsilon = 0.7$, $\alpha = 1$, $\ell_{min} = 10^{-12}\,\mathrm{m}$ and $\kappa_M^{max} = 100\,\mathrm{m}^2/\mathrm{s}$. The floors κ_M^{min} and κ_H^{min} of viscosity and diffusivity are zero by default, as in pyOM2. With the TKE closure on, the constant coefficients A_v and κ of the hierarchy above are not used. A background viscosity or diffusivity is therefore a choice of the configuration. It is made by setting the floors. Note that the pyOM2 configurations of the acc family deviate from the module defaults and set $\alpha = 30$ (a proxy for vertical TKE transport by pressure work, which spreads the surface-injected TKE substantially deeper) and $\ell_{min} = 10^{-8}\,\mathrm{m}$; for consistency with those experiments both should be set accordingly (`alpha_tke`, `mxl_min`). The turbulent kinetic energy e is a prognostic variable on the W points,

$$\partial_t e = \partial_z\left(\alpha\,\kappa_M \partial_z e\right) + K_{diss} - P_{diss} - c_\epsilon\, e^{3/2}/\ell \tag{23}$$

with the mixing length $\ell = \sqrt{2e}/N$, limited such that it cannot grow faster than the distance to the boundaries (two sweeps, pyOM2's `tke_mxl_choice=2`), and the diffusivities

$$\kappa_M = \min(\kappa_M^{max},\, c_k\,\ell\sqrt{e}), \qquad \kappa_H = \kappa_M/\max(1, \min(10,\, 6.6\,Ri)), \qquad Ri = N^2\kappa_M/K_{diss} \tag{24}$$

There is no separate convective branch. Where $N^2 < 0$ the Richardson number is negative. The Prandtl number is then one and $\kappa_H = \kappa_M$, exactly as in pyOM2. This replaces the convective-adjustment option. The ratio K_{diss}/κ_M in Ri is the squared vertical shear of the friction operator. With `mix_eulerian` (see below) it is $\partial_z\bar{\boldsymbol{u}} \cdot \partial_z\left(\bar{\boldsymbol{u}} + \boldsymbol{u}_S\right)$. Without it, it is the squared shear of the model velocity. N^2 lives on the W points, i.e. on the top face of each cell. At the surface face there is no cell above, so N^2 is copied from the face below, as in pyOM2. For `eq_of_state` $= 5$ the density of the upper cell is evaluated at the pressure of the lower cell and compared with the density of the lower cell (adiabatic displacement, as in pyOM2).

Two sets of limits for the mixing length are available. They are selected by `mxl_choice`. The default `mxl_choice` $= 1$ is the pyOM2 form named above. Its sweeps run in the order of `tke.f90`. First a downward sweep from the surface limits $\ell_k \le \ell_{k+1} + \Delta z$. It starts from the unlimited value at the surface face. Then the value at the surface face is limited to $\ell \le \ell_{min} + z_0 + \Delta z$, with the roughness length z_0 given below. Finally an upward sweep from the bottom limits $\ell_k \le \ell_{k-1} + \Delta z$. The same length enters the diffusivity and the dissipation. With `mxl_choice` $= 2$ the limits of NEMO 4.2 are used (`nn_mxl` $= 3$ with `ln_mxl0`). All lengths live on the W points, i.e. at the top face of each tracer cell. The index n counts these points upward. Here $n = 1$ is the lowest point of the array and n_s the surface face. The starting point is the buoyancy length

$$\ell_n = \max\left(\sqrt{2e}/\sqrt{\max(N^2, 10^{-12}\,\mathrm{s}^{-2})},\ \ell_{min0}\right)$$

with $\ell_{min0} = 0.04\,\mathrm{m}$ (attribute `mxl_min0`). This floor is applied at all W points, also at dry points where $e = 0$. The surface value is then reduced to the surface length,

$\ell_{n_s} = \min(\ell_{n_s}, \ell_0)$, with ℓ_0 given below. Since ℓ_0 is set before the downward sweep, it bounds the whole near-surface profile and not only the surface point. Two profiles are formed from ℓ_n. Going downward from the surface the length is limited by the distance to the surface,

$$\ell^{up}_n = \min\left(\ell_n,\ \ell^{up}_{n+1} + \Delta z\right), \quad n = n_s - 1, \ldots, 1, \qquad \ell^{up}_{n_s} = \ell_{n_s}$$

Going upward from the bottom of the array the length is limited by the distance to the bottom,

$$\ell^{dwn}_n = \min\left(\ell_n,\ \ell^{dwn}_{n-1} + \Delta z\right), \quad n = 2, \ldots, n_s, \qquad \ell^{dwn}_1 = \ell_1$$

The two sweeps run over the whole array, dry points included. In a column whose lowest wet cell lies above the bottom of the array, the W point below the lowest wet cell is dry. There $\ell_n = \ell_{min0}$, so the lowest wet point gets $\ell^{dwn} \le \ell_{min0} + \Delta z$. In a column that is wet down to the bottom of the array the lowest point gets $\ell^{dwn}_1 = \ell_1$ without this limit. The diffusivity uses the geometric mean of the two profiles and the dissipation uses their minimum,

$$\ell_k = \sqrt{\ell^{up}\,\ell^{dwn}}, \qquad \ell_\epsilon = \min\left(\ell^{up},\ \ell^{dwn}\right)$$

ℓ_k replaces ℓ in the diffusivity κ_M, and ℓ_ϵ replaces ℓ in the dissipation $c_\epsilon e^{3/2}/\ell$. The constant c_ϵ multiplies the dissipation here. Writing the dissipation as $e^{3/2}/\ell_\epsilon$ with $\ell_\epsilon = \min(\ell^{up}, \ell^{dwn})/c_\epsilon$ is the same. The surface length ℓ_0 has two forms. Without a roughness length it follows Charnock's relation

$$\ell_0 = \kappa_{vk}\,\beta\,|\boldsymbol{\tau}|/g$$

with the kinematic surface stress $\boldsymbol{\tau}$, i.e. the stress divided by ρ_0, the von Kármán constant $\kappa_{vk} = 0.4$ (attribute `mxl_kappa`) and the Charnock constant β (attribute `mxl_beta`). The default $\beta = 2 \cdot 10^5$ is the value of NEMO. It gives $\ell_0 = 0.8\,\mathrm{m}$ at a stress of $0.1\,\mathrm{Pa}$. The value $\beta = 2 \cdot 10^4$ gives $\ell_0 = 0.08\,\mathrm{m}$ at the same stress. The modulus $|\boldsymbol{\tau}|$ is formed pointwise from `taux` and `tauy`, or taken from `tau_modulus` if the configuration provides it. With a roughness length $z_0 > 0$ (attribute `z0_surf`) the surface length is $\ell_0 = \kappa_{vk}\,z_0$ instead. The coupled configurations set $z_0 = 1.6\,H_s$ with the significant wave height H_s from the wave model. In both forms $\ell_0 \ge \ell_{min0}$. One halo exchange of ℓ_0 is made per time step. A configuration whose stress carries valid halo values can avoid it by setting `tau_halo_ok`.

The discretisation follows pyOM2: backward Euler with the dissipation linearised in the diagonal ($c_\epsilon\sqrt{e^n}/\ell$ acting on e^{n+1}), a surface W cell of half thickness $\Delta z/2$, and the surface flux boundary condition added to that cell,

$$F_e(0) = \begin{cases} \Phi_{oc} & \text{wave dissipation flux from the wave model} \\ (|\boldsymbol{\tau}|)^{3/2} & \text{classical parameterisation} \end{cases} \tag{25}$$

selected by `tke_surf_from_waves` (default on, falling back to the classical form if no wave flux is present; $\boldsymbol{\tau}$ kinematic; the first form closes the energy cycle between the wave and

ocean components, the second is the pyOM2 default and serves as the control experiment). The classical form carries the factor one of pyOM2. Craig and Banner (1994) give $\alpha_{cb}|\boldsymbol{\tau}|^{3/2}$ with $\alpha_{cb} \approx 100$. pe_simple has no attribute for this factor. A configuration that wants it overloads `_surface_forcing`, as `setups/coupled_common/exp_ocean.py` does with `EXP_ALPHA_CB`. No diffusive TKE, tracer or momentum flux passes through the surface face itself: the surface forcing enters only as the explicit flux above, which also removes a spurious shear production against the zero-velocity halo above the surface (a phantom no-slip lid). For wave-driven injection a surface roughness length z_0 can be set (attribute `z0_surf`, default 0, e.g. $1.6\,H_s$ from the wave model in the sense of Mellor & Blumberg 2004). With `mxl_choice` $= 1$ it only raises the limit of the value at the surface face. The downward sweep runs before that limit is applied, so the mixing length below the surface face does not change. With `mxl_choice` $= 2$ it sets the surface length ℓ_0 described above. If the surface cell is drained negative by buoyancy fluxes, the missing energy is added back and booked in `tke_surf_corr` as in pyOM2. There is no other floor on e, as in pyOM2. A floor $e \geq e_{min}$ is applied only if `tke_min` > 0 is set. The initial value of e in all wet cells is `tke_init` $= 10^{-12}\,\mathrm{m}^2/\mathrm{s}^2$, as in the pyOM2 configurations.

Vertical friction is computed implicitly with κ_M averaged to the U/V points as in pyOM2 ($\overline{\kappa_M}^i$). A momentum flux exists only across a face between two wet U (or V) cells, as in pyOM2. TKE is likewise exchanged only between two wet W levels. The shear production is diagnosed from the same discrete operator with the updated velocities,

$$K_{diss} = \overline{\overline{\kappa_M}^i \left(\partial_z u^{n+1}\right)^2}^i + \overline{\overline{\kappa_M}^j \left(\partial_z v^{n+1}\right)^2}^j \tag{26}$$

The backward-Euler friction step is $\boldsymbol{u}^{n+1} - \boldsymbol{u}^n = \Delta t\, \partial_z \left(\overline{\kappa_M}\, \partial_z \boldsymbol{u}^{n+1}\right)$. Multiplying it by $\boldsymbol{u}^{n+1}$ and integrating over the column gives the kinetic energy it removes,

$$\int_{-h}^{0} \frac{|\boldsymbol{u}^n|^2 - |\boldsymbol{u}^{n+1}|^2}{2\Delta t}\, dz = \int_{-h}^{0} \overline{\kappa_M} \left(\partial_z \boldsymbol{u}^{n+1}\right)^2 dz + \int_{-h}^{0} \frac{|\boldsymbol{u}^{n+1} - \boldsymbol{u}^n|^2}{2\Delta t}\, dz$$

The first term on the right is K_{diss} and is the forcing of the TKE equation. The second term is the numerical dissipation of the backward-Euler step. It is of order Δt and is not put into the TKE equation. The kinetic energy removed from the mean flow therefore reappears in the TKE forcing up to this term. $P_{diss} = \kappa_H N^2$ is the exact transfer of the implicit buoyancy mixing, evaluated with the *updated* buoyancy (the tracer solve precedes the TKE step; using the pre-mixing N^2 over-drains the TKE whenever $\kappa_H \Delta t/\Delta z^2$ is not small). At the surface face P_{diss} is the surface buoyancy flux instead, as in pyOM2,

$$P_{diss}(0) = -\frac{g}{\rho_0}\left(\frac{\partial \rho}{\partial T} F_T + \frac{\partial \rho}{\partial S} F_S\right)$$

Here F_T and F_S are the surface fluxes of temperature and salt into the ocean (K m/s and psu m/s, positive into the ocean). The configuration provides them as the attributes `forc_temp_surface` and `forc_salt_surface`. Surface cooling is thus a source of TKE. Without these attributes the term is zero. For `eq_of_state` $= 0$ the temperature is the

buoyancy b and $\partial\rho/\partial b = -\rho_0/g$ is used. pyOM2 uses zero there and has no surface buoyancy source of TKE for this equation of state. The surface W cell has the thickness $\Delta z/2$. The TKE that enters it per unit area and time is therefore $F_e(0)+(\Delta z/2)\,J_s$ plus the correction `tke_surf_corr` described above. Here $J_s = -P_{diss}(0)$ is the surface buoyancy loss. In stand-alone column tests the discrete TKE budget (surface flux + shear production = dissipation + buoyancy destruction + storage) closes to machine precision.

When the model velocity is the Lagrangian mean $\boldsymbol{u}_L$ of the coupled wave–ocean formulation, the attribute `mix_eulerian` (default off, requires the Stokes velocity to be present) lets the friction operator act on the Eulerian mean velocity $\bar{\boldsymbol{u}} = \boldsymbol{u}_L - \boldsymbol{u}_S$: the down-gradient closure parameterises Eulerian fluctuations, so mixing $\boldsymbol{u}_L$ would spuriously diffuse the Stokes shear. The Stokes velocity is subtracted before the implicit solve and added back afterwards, and the shear-production booking becomes flux times *Lagrangian* shear, split into the non-negative Eulerian part $K^E_{diss} = \kappa_M(\partial_z\bar{\boldsymbol{u}})^2$ and the sign-indefinite Stokes (Langmuir) part $K^L_{diss} = \kappa_M\partial_z\bar{\boldsymbol{u}}\cdot\partial_z\boldsymbol{u}_S$, kept separately in the diagnostics with $K_{diss} = K^E_{diss} + K^L_{diss}$ feeding the TKE equation. Without `mix_eulerian` this reduces bit-identically to the plain booking above.

The calling sequence per time step is: diffusivities from the current e and N^2 at the start of the step (so buoyancy mixing and friction use the same coefficients the transfers are paired with), vertical friction after the pressure correction, and the TKE step at the end of the time step. The density is evaluated once per time step, as in pyOM2. This happens after the vertical mixing of the tracers and before the TKE step. N^2 and P_{diss} of the TKE step use this updated density. The hydrostatic pressure of the next time step is computed from the same density.

Model parameters and options

All parameters and options are attributes of the model instance, set either through the parameter dictionary passed at construction or by assigning the attribute before `prepare_for_run`; options with a default may be left unset. The tables list every attribute the model reads.

Required parameters (no default)

Attribute	Unit	Meaning
`nx`, `ny`, `nz`	—	global grid size (cells, without halo points)
`dx`, `dy`, `dz`	m	grid spacing; Δz is a uniform scalar
`dt`	s	time step

Attribute	Unit	Meaning
`eq_of_state`	—	equation of state: 0 (temperature carries the buoyancy, no salinity), 1 (linear $\rho(S,T)$, salinity prognostic), 5 (GSW/TEOS-10, salinity prognostic, the pressure is the depth of the cell centre below the surface in dbar), 100 (ideal-gas atmosphere)
`implicit_free_surface`	—	implicit free surface if true
`explicit_free_surface`	—	split-explicit free surface if true; both false: rigid lid
`periodic_in_x`, `periodic_in_y`	—	lateral boundaries periodic (true) or closed (false)
`epsilon`	—	convergence threshold of the iterative surface-pressure solver (rigid lid / implicit free surface)
`max_iterations`	—	iteration cap of that solver
`sub_cycling`	—	number of barotropic sub-cycles M (split-explicit scheme only)

The constructor additionally takes `n_pes_x`, `n_pes_y` (processor grid, default 1×1), `halo_points` (default 1, at least 3 for `weno_advection`) and `comm_in` (default: MPI world communicator). The constants `grav` = 9.81 and `rho_0` = 1024 are set internally (note: they overwrite any value passed in the parameter dictionary). Coriolis parameter, topography (`kbot`) and initial conditions are set through the template methods `set_coriolis`, `set_topography` and `set_initial_conditions`, overloaded in the configuration.

Forcing hooks and coupling fields

Attribute / method	Meaning
`set_forcing(tau)`	optional user method; adds forcing to the momentum tendencies `du[tau]`, `dv[tau]` (enters the AB interpolation). Surface fluxes of heat and salt are added to `dtemp[tau]` and `dsalt[tau]` in the same way. Both schemes keep these tracer increments
`set_forcing_direct()`	optional user method; direct momentum increment bypassing the AB weights, applied before the pressure solve. Used for the Stokes tendency $\Delta \boldsymbol{u}_S$ of the Lagrangian-mean ($\boldsymbol{u}_L$) formulation, which must be a plain forward increment. It is called only in the rigid-lid and implicit free-surface scheme. The split-explicit scheme does not call it
`forc_temp_surface`	surface flux of temperature into the ocean (K m/s). It gives the surface buoyancy source $P_{diss}(0)$ of the TKE equation. It does not force the tracer. That is done in `set_forcing`

Attribute / method	Meaning
`forc_salt_surface`	the same for salt (psu m/s)
`tau_modulus`	optional modulus of the kinematic surface stress at the T points, used by `mxl_choice` = 2 instead of the pointwise modulus of `taux`, `tauy`
`taux`, `tauy`	kinematic wind stress; also the fallback surface TKE forcing $\lvert\boldsymbol{\tau}\rvert^{3/2}$
`phi_oc`	wave-breaking energy flux from the wave model; surface TKE flux when `tke_surf_from_waves` is on
`u_S`, `v_S`	Stokes velocity from the wave model; required by `mix_eulerian`

Options with defaults

Attribute	Default	Meaning
`superbee_advection`	unset	if set true, tracer advection uses the Superbee flux limiter instead of the second-order fluxes (momentum advection is always second order). Note: setting it to false raises an error; leave it unset for the second-order scheme
`weno_advection`	off	tracer advection with the fifth-order WENO scheme, see the section on tracer advection. It needs `halo_points` = 3. It takes precedence over `superbee_advection`
`weno_ref`	1	size T_{ref} of the tracer in $\epsilon = 10^{-8}T_{ref}^2$ of the WENO weights, in the unit of the tracer, the same for temperature and salinity
`Khbi`	unset	biharmonic horizontal tracer diffusion coefficient (m^4/s); active if present, forward Euler

Attribute	Default	Meaning
`Ahbi`	unset	biharmonic horizontal friction coefficient (m^4/s); active if present, forward Euler. In the rigid-lid and implicit free-surface scheme it is applied to $\boldsymbol{u}$ before the AB update. In the split-explicit scheme it is a tendency in the forcing $\boldsymbol{R}^n$ of the external mode
`vmix_scheme`	`'explicit'`	vertical mixing/friction hierarchy: `'explicit'` (forward Euler, coefficient checked against the diffusive stability limit) or `'implicit'` (backward Euler, unconditionally stable). If unset, the legacy flag `enable_implicit_vmix` decides
`kappa_T`	0	background vertical diffusivity (m^2/s)
`A_v`	0	background vertical viscosity (m^2/s)
`enable_conv_adjust`	on iff implicit	convective adjustment: $\kappa = \kappa_{conv}$ where $N^2 < 0$ (implicit scheme only)
`kappa_conv`	1	convective-adjustment diffusivity (m^2/s)
`enable_tke`	off	TKE closure; overrides the constant coefficients for both buoyancy and momentum (implicit scheme only)
`enable_bottom_friction`	off	linear bottom friction on the lowest wet cell, rate `r_bot`
`r_bot`	0	rate of the linear bottom friction ($1/s$)
`enable_quadratic_bottom_friction`	off	quadratic bottom friction on the lowest wet cell, rate $C_d\|\boldsymbol{u}_b\|/\Delta z$
`r_quad_bot`	0	drag coefficient C_d of the quadratic bottom friction
`fb_theta`	0.14	dissipation parameter θ of the split-explicit sub cycle

Attribute	Default	Meaning
`free_surface_tracer_flux`	off	tracer flux through the surface face. Off means zero flux, as in pyOM2. On means $w\,b$ with the surface-cell value, so a uniform tracer stays uniform

TKE closure constants (read at `init_tke`)

Attribute	Default	Meaning
`c_k`	0.1	diffusivity constant, $\kappa_M = c_k \ell \sqrt{e}$
`c_eps`	0.7	dissipation constant, $c_\epsilon e^{3/2}/\ell$
`alpha_tke`	1	vertical TKE diffusion $\alpha\kappa_M$; the pyOM2 acc configurations use 30
`mxl_min`	10^{-12}	mixing-length floor (m); acc configurations use 10^{-8}
`kappaM_min`	0	viscosity floor (m^2/s), as in pyOM2. A background viscosity is set here
`kappaM_max`	100	viscosity cap (m^2/s)
`kappaH_min`	0	diffusivity floor (m^2/s). A background diffusivity is set here
`tke_min`	0	TKE floor (m^2/s^2), applied only if positive
`tke_init`	10^{-12}	initial TKE in wet cells (m^2/s^2)
`mxl_choice`	1	limits of the mixing length. 1 is pyOM2, 2 is NEMO 4.2 with a separate length for the dissipation
`mxl_kappa`	0.4	von Karman constant of the surface length (`mxl_choice` = 2)
`mxl_beta`	$2 \cdot 10^5$	Charnock constant β of the surface length (`mxl_choice` = 2)
`mxl_min0`	0.04	minimum surface and bottom length (m) (`mxl_choice` = 2)
`tau_halo_ok`	off	set when `taux`, `tauy` carry valid halo values. Saves the halo exchange of the surface length (`mxl_choice` = 2)
`z0_surf`	0	surface roughness length (m), e.g. $1.6\,H_s$. Limits the value at the surface face (`mxl_choice` = 1) or sets the surface length (`mxl_choice` = 2)
`tke_surf_from_waves`	on	surface TKE flux from the wave dissipation Φ_{oc}; falls back to $\lvert\boldsymbol{\tau}\rvert^{3/2}$ if no wave flux is present

Attribute	Default	Meaning
`mix_eulerian`	off	vertical friction acts on the Eulerian mean velocity $\boldsymbol{u}_L - \boldsymbol{u}_S$ ($\boldsymbol{u}_L$ formulation), with the K^E_{diss}/K^L_{diss} booking split

Restart state

The model has no restart routine of its own. A configuration that continues a run must save the following fields and set them again before the next step. A restart is then bit-identical to the uninterrupted run. This is tested by `pyOM3/test/pe/restart.py`, which holds the reference list.

- Always the fields `u`, `v`, `w`, `temp`, `salt`, `rho`, `p_h` and `p_s`.
- Always the three time levels of the tendencies `du`, `dv`, `dtemp` and `dsalt`.
- The time-level pointers `tau`, `taum1`, `taum2` and the AB coefficients of the driving loop. The method `loop` starts every call with a forward Euler step and the pointers $2, 1, 0$. A restart must therefore drive the time steps itself.
- With the TKE closure `tke` and `K_diss_v`. The diffusivities of a step are built from K_{diss} of the previous step through the Richardson number. Without `K_diss_v` the first step after the restart differs.
- With the split-explicit scheme `du_cor`, `dv_cor`, `U_bt` and `V_bt`.
- With `mxl_choice` = 2 `taux` and `tauy` (or `tau_modulus`). The closure uses them at the start of the step, before `set_forcing` sets the new stress.

Snapshot diagnostics

Two snapshot classes are available; either replaces the model class as the base of the configuration and overrides `prepare_for_run` and `diagnose`. Both read the attribute `snapint` (snapshot interval in seconds, required) and take the `prepare_for_run` arguments `snap_file_name` and `show_each_time_step` (default on: print every time step; off: print at output steps only). When the iterative surface-pressure solver is active, both print its per-step iteration count and accumulate the total count between outputs.

snap_nc4 writes a single netCDF file (default name `'snap.cdf'`), created with the T- and U-grid coordinate axes and the variables u, v, w, temp, p_h, ρ and p_s; salinity is included when the equation of state carries it (`eq_of_state` > 0 and $\neq 100$). At every output step each PE in turn writes its halo-free sub-block into the global arrays of the shared file (serialised with barriers). For the rigid-lid/implicit path the current time level of the surface pressure is written, for the split-explicit path the plain 2D field. This class works with the MPI domain decomposition and with single-process JAX, but not in a multihost JAX environment, since all processes must be able to open the same file.

snap_raw writes one `.npy` file per variable, per PE and per output step into a directory tree `<name>/pe=<rank>/n=<step>/` (default name `'raw_snap'`; an existing directory of that name is removed at start-up), with the coordinate axes and the processor-grid layout (`decomposition.npy`) stored once at the top level. Each process only touches its own subdirectory, and JAX sharded arrays are written through their local shard, so this class also works in a multihost JAX environment. The variable set is u, v, w, temp, salt, p_h and p_s (salt always, ρ not included). The fragmented output is meant to be reassembled afterwards using the stored decomposition, e.g. into zarr.